

Análise de Incidente: Dupla Reserva por Race Condition

Data: 2026-06-04 | Severidade: Alta | Sistema: Hotel Fácil — Serviço de Reservas

Descrição

Dois clientes conseguiram reservar o mesmo quarto para o mesmo período. O quarto 101 do Hotel Atlântico foi reservado duas vezes para 01/08/2026 a 05/08/2026, e ambos receberam confirmação com HTTP 201.

Logs

As duas requisições foram disparadas simultaneamente. Abaixo o que foi registrado:

```
2026-06-04T02:03:23.661 INFO [http-nio-8080-exec-3]
ReservationServiceImpl:
```

```
    Creating reservation roomId=1 guest='Cliente B'
    checkIn=2026-08-01 checkOut=2026-08-05
```

```
2026-06-04T02:03:23.661 INFO [http-nio-8080-exec-4]
ReservationServiceImpl:
```

```
    Creating reservation roomId=1 guest='Cliente A'
    checkIn=2026-08-01 checkOut=2026-08-05
```

2026-06-04T02:03:23.674 DEBUG [http-nio-8080-exec-3]
ReservationServiceImpl:

Found 4 available rooms for hotelId=1

2026-06-04T02:03:23.674 DEBUG [http-nio-8080-exec-4]
ReservationServiceImpl:

Found 4 available rooms for hotelId=1

2026-06-04T02:03:23.678 INFO [http-nio-8080-exec-3]
ReservationServiceImpl:

Reservation created id=2 room=101 guest='Cliente B'
total=1400.00

2026-06-04T02:03:23.678 INFO [http-nio-8080-exec-4]
ReservationServiceImpl:

Reservation created id=3 room=101 guest='Cliente A'

```
total=1400.00
```

Evidência no banco após o incidente:

```
id=2  quarto=101  hóspede='Cliente B'  checkIn=2026-08-01  
checkOut=2026-08-05
```

```
status=PENDING  createdAt=2026-06-04T05:03:23.678788Z
```

```
id=3  quarto=101  hóspede='Cliente A'  checkIn=2026-08-01  
checkOut=2026-08-05
```

```
status=PENDING  createdAt=2026-06-04T05:03:23.678803Z
```

```
Diferença entre os createdAt: 15 microssegundos.
```

Causa Raiz

O fluxo de criação de reserva executa a verificação de disponibilidade e a inserção como duas operações separadas, sem exclusão mútua entre threads:

```
exec-3 (Cliente B): verifica disponibilidade -> quarto 101  
livre
```

```
exec-4 (Cliente A): verifica disponibilidade -> quarto 101
livre (exec-3 ainda não salvou)

exec-3 (Cliente B): salva reserva id=2

exec-4 (Cliente A): salva reserva id=3 <- duplicata criada
```

O problema é clássico de TOCTOU (Time-of-Check to Time-of-Use): o estado do recurso é lido em um momento e utilizado em outro, sem garantia de que nada mudou entre os dois.

Impacto

Dois hóspedes com reserva confirmada para o mesmo quarto. Operacionalmente exige cancelamento manual e realocação. Dependendo da situação pode gerar reembolso e dano à reputação do serviço.

Correções Sugeridas

1. Lock pessimista no banco

Ao buscar o quarto antes de criar a reserva, usar *SELECT ... FOR UPDATE*. Isso bloqueia o registro até o fim da transação, impedindo que outra thread leia o quarto como disponível enquanto a primeira ainda está processando.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)

@Query("SELECT r FROM Room r WHERE r.id = :id")

Optional<Room> findByIdForUpdate(@Param("id") Long id);
```

1. Unique constraint no banco

Adicionar uma constraint de unicidade na tabela de reservas como segunda camada de defesa. Mesmo que o lock falhe, o banco rejeita a inserção duplicada.

```
ALTER TABLE reservations

ADD CONSTRAINT uq_room_period UNIQUE (room_id,
check_in, check_out);
```

A aplicação deve capturar o `DataIntegrityViolationException` e retornar 409 Conflict.

1. Idempotency key

Aceitar um header *Idempotency-Key* nas requisições de reserva. O backend armazena a chave e, ao receber uma segunda requisição com a mesma chave, retorna o resultado já existente sem processar novamente.

Prevenção

Adicionar testes de concorrência cobrindo esse cenário, disparando múltiplas threads simultâneas para o mesmo quarto e verificando que apenas uma reserva é criada.

Implementar alerta automático quando duas reservas com o mesmo quarto e período sobrepostos forem persistidas no banco.

No frontend, tratar 409 Conflict informando ao usuário que o quarto acabou de ser reservado.

Linha do Tempo

05:03:23.661 Duas requisições chegam simultaneamente

05:03:23.674 Ambas consultam disponibilidade – quarto aparece livre nas duas

05:03:23.678 Reserva id=2 criada (Cliente B)

05:03:23.678 Reserva id=3 criada (Cliente A) – 15 microssegundos depois

05:03:23.678 Ambas retornam HTTP 201